

PERFORMANCE EVALUATION OF GRAPHICS API MIGRATION FROM OPENGL TO DIRECTX IN REAL-TIME VOLCANIC PLUME VISUALIZATION

A Thesis Proposal

presented to

the Department of Software Technology
College of Computer Studies
De La Salle University

In partial fulfillment
of the requirements for the degree of

Bachelor of Science in Computer Science
Major in Software Technology

by

BERNANDINO, Adrian Rafael
BORRAMEO, Anton Miguel
GALAN, Russell Emmanuel
GANAYO, Reinman Geoffrey

Neil DEL GALLEGO
Adviser

July 3, 2026

Abstract

Effective real-time visualization of dynamic natural hazards like volcanic eruptions plays a crucial role in disaster preparedness and public education, yet traditional scientific simulations often compromise real-time interactivity due to high computational overhead. While existing interactive tools like the OpenGL-based AnitoPlume simulator achieve real-time rendering, their reliance on high-level graphics APIs introduces significant CPU driver overhead and limits explicit control over modern GPU resources. This study addresses the challenge of modernizing scientific visualization systems by migrating the rendering subsystem of AnitoPlume to a low-level explicit graphics API, exploring whether the increased implementation complexity yields practical architectural and performance benefits. The primary contribution of this research is a modernized, DirectX-based rendering framework for the AnitoPlume simulator that preserves original physical behaviors and visual output while exposing direct hardware control. To achieve this, the methodology involves systematically porting the rendering pipeline, shader programs, and memory management structures from OpenGL to DirectX, followed by a comparative performance evaluation under identical hardware configurations. This research establishes an empirical baseline for graphics API migration in scientific visualization, providing a highly scalable foundation for integrating modern hardware-accelerated rendering techniques.

Keywords: AnitoPlume, real-time visualization, DirectX, OpenGL, graphics API migration

Contents

1	Introduction	1
1.1	Background of the Study	1
1.2	Research Objectives	2
1.3	Scope and Limitations of the Research	3
1.4	Significance of the Research	4
2	Related Works	5
2.1	Test	5
2.2	AnitoPlume	5
2.2.1	Graphics Architecture and System Performance	5
2.2.2	Usability Design and Interactive Features	6
2.2.3	System Validation and Evaluation Metrics	6
2.2.4	Architectural Limitations and Future Trajectories	6
2.3	Graphics API Architecture and Migration Implications	7
2.3.1	OpenGL Implicit State vs. DirectX 12 Explicit State	7
2.3.2	Exploding the CPU Command Recording Bottleneck	8
2.3.3	Explicit Memory Management Resource Binding	8
2.3.4	Falcor Rendering Framework	8

Chapter 1

Introduction

1.1 Background of the Study

Volcanic eruptions are among the most destructive natural hazards, producing ash plumes, pyroclastic flows, and volcanic gases that threaten nearby communities and critical infrastructure. Effective visualization of these hazards plays an important role in disaster preparedness, scientific communication, and public education. Traditionally, volcanic simulations have relied on numerical models that prioritize scientific accuracy for hazard prediction and analysis. While these models provide valuable quantitative information, they are often computationally intensive and lack the interactive capabilities necessary for real-time visualization (?, ?).

To overcome these limitations, the sub-field of real-time computer graphics has increasingly been applied to scientific visualization, balancing visual plausibility with high interactive performance. Within this domain, a notable localized development is AnitoPlume, a real-time three-dimensional volcanic plume simulator designed specifically for visualizing the complex eruption dynamics of Taal Volcano (?, ?). Developed using C++ and the OpenGL graphics API, this specialized application combines a Lagrangian particle-based plume system with a detailed digital elevation model of the volcano's terrain. By maintaining an interactive rendering performance of approximately 30 frames per second, the system successfully demonstrated that computer graphics could bridge the gap between complex geological events and accessible hazard awareness software for local education and planning (?, ?).

Previous efforts in graphics-driven volcanic visualization have heavily relied

on traditional higher-level APIs like OpenGL due to their broad driver portability and structural simplicity (?). In these legacy architectures, the graphics API simplifies software development by delegating critical hardware management tasks—such as memory allocation and command scheduling—directly to the vendor’s graphics driver. However, as visualization workloads scale, this abstraction inevitably introduces massive CPU driver overhead and restricts developers from optimizing how the hardware executes commands (?). Consequently, the computer graphics industry has seen a global shift toward modern, low-level explicit graphics APIs like DirectX (?), which strip away driver intervention and grant developers granular, direct control over GPU resources, pipeline states, and memory synchronization to maximize hardware efficiency (?).

Despite the theoretical performance advantages offered by modern explicit rendering pipelines, a significant challenge remains in the software modernization of established scientific tools. High-level visualization platforms like the original AnitoPlume simulator are tightly coupled to OpenGL architectures, creating a rigid barrier to optimization and preventing the integration of advanced, contemporary rendering techniques. Transitioning an active simulation system to a low-level framework like DirectX introduces extreme architectural complexity, requiring a complete restructuring of shader programs, command submissions, and memory allocation strategies (?). It remains an open question whether the practical rendering efficiency, resource utilization, and extensibility gains of such a migration justify the steep technical overhead required to port real-time hazard visualization software without altering its underlying scientific behaviors.

1.2 Research Objectives

The primary aim of this study is to modernize the rendering architecture of the AnitoPlume volcanic plume simulator by porting its OpenGL implementation to DirectX and evaluating the impact of the migration on rendering performance, system architecture, and future extensibility.

The specific objectives of this research are as follows:

1. To port the rendering subsystem of the published AnitoPlume simulator from OpenGL to DirectX 12 while strictly preserving its baseline simulation functionality and visual output;
2. To identify and document the technical engineering challenges encountered during the migration process, focusing on shader translation, rendering

pipeline state adaptation, explicit resource management, and command synchronization.

3. To compare the performance of the OpenGL and DirectX 12 implementations under identical hardware environments using empirical metrics, including frame rate (FPS), CPU/GPU resource utilization, and frame rendering times; and
4. To assess the architectural maintainability and extensibility of the new DirectX 12 foundation for integrating future hardware-accelerated rendering techniques into the simulator.

1.3 Scope and Limitations of the Research

This study is bounded by the technical objectives of migrating an existing graphics application to a modern architecture, establishing strict constraints to isolate rendering engine efficiency.

The architectural analysis is limited to comparing the state-machine model of OpenGL with the explicit, low-overhead pipeline model of DirectX 12 within real-time scientific visualization frameworks. This evaluation is strictly constrained to the Windows operating system environment. It excludes alternative modern APIs like Vulkan or Metal because the baseline AnitoPlume application is native to Windows, justifying DirectX 12 as the most appropriate target platform for native hardware acceleration.

The implementation phase is strictly confined to porting the rendering subsystem of the published AnitoPlume simulator from OpenGL to DirectX 12. The scope includes migrating existing graphics resources, translating shader programs into High-Level Shader Language (HLSL), and rewriting command buffers to accurately reproduce the original terrain and particle rendering. This study explicitly excludes any modifications to the core scientific simulation algorithms, Lagrangian physics math, or the terrain mesh generation developed by Del Gallego et al. This boundary ensures that the physical simulation remains a controlled variable, preventing data contamination during the performance comparison.

The documentation of technical challenges is focused entirely on low-level engineering hurdles native to explicit API migration, such as pipeline state object (PSO) compilation and explicit memory synchronization. The study excludes user-facing software usability, user interface design adjustments, or human participant testing, as application usability was already verified in the baseline

study. Instead, data sources are purely technical, derived from automated profiling tools (e.g., PIX or RenderDoc) capturing frame rate (FPS), rendering times, and CPU/GPU utilization during a standardized 180-second simulated eruption sequence. Testing is bounded to standard consumer-grade discrete graphics hardware to reflect the deployment environment of local disaster-response centers.

Finally, the assessment of future extensibility is limited to structural code evaluations and modularity analysis regarding how the new DirectX 12 architecture prepares the simulator to support contemporary rendering pipelines. The actual implementation of advanced features, such as hardware-accelerated ray tracing or compute shaders, is excluded from this study to avoid scope creep while providing a clear architectural roadmap for future researchers.

1.4 Significance of the Research

This study contributes to both computer graphics research and scientific visualization by examining the modernization of an existing real-time simulation system rather than developing an entirely new application. Through the successful migration of AnitoPlume from OpenGL to DirectX, the research provides practical insights into graphics API migration and software modernization for visualization systems.

For researchers in computer graphics, the study offers a detailed case study demonstrating the architectural modifications required when transitioning between two fundamentally different graphics APIs. The comparison between the OpenGL and DirectX implementations contributes empirical evidence regarding rendering performance, implementation complexity, and software maintainability.

For developers of scientific visualization software, the findings may serve as a reference for future projects that require modernization of legacy rendering systems while preserving existing simulation logic. The documented migration process may also assist developers considering similar transitions to lower-level graphics APIs.

For future researchers, this work establishes a modern DirectX-based rendering foundation that may support the integration of advanced graphics techniques, including hardware-accelerated ray tracing, compute shaders, GPU-driven rendering, and other contemporary visualization methods. Consequently, the study extends the long-term applicability of the original AnitoPlume project beyond its initial OpenGL implementation.

Chapter 2

Related Works

2.1 Test

2.2 AnitoPlume

AnitoPlume is the first interactive, real-time 3D graphics-based simulation application explicitly tailored for visualizing the changing landscape of the Taal Volcano island in the Philippines and its corresponding eruption plumes. Developed primarily to address the challenges of volcanic hazard and crisis communication, the simulator serves as an immersive, lightweight alternative to traditional 2D heatmaps and highly complex, non-interactive geoscientific numerical solvers (such as FALL3D and Tephra2). While geoscientific solvers excel at quantitative predictive accuracy, they require heavy high-performance computing resources and significant processing time, making them unviable for applications requiring immediate visual feedback, interactive flexibility, and low-latency user immersion. AnitoPlume fills this gap by merging computer graphics techniques with site-specific real-world terrain details to facilitate risk awareness and "what-if" situational modeling (?, ?).

2.2.1 Graphics Architecture and System Performance

To bypass the overhead, heavy abstractions, and unneeded general-purpose features of commercial game engines, AnitoPlume was custom-built using a lean, native C++ open-source API stack consisting of OpenGL for graphics render-

ing, Eigen for numerical linear algebra, and dearImGui for the user interface layout. This custom application architecture optimizes real-time performance on consumer-grade hardware. By keeping the system dependencies minimal, the simulator successfully achieves real-time interactivity, running at an average rendering rate of 30 frames per second (FPS) on a mid-range system with 6 GB of video RAM (VRAM) (? , ?).

2.2.2 Usability Design and Interactive Features

The simulator also introduced several usability improvements over previous volcanic visualization systems, including simplified camera controls, interactive parameter editing, real-time plume direction tracking, and contextual information about nearby communities affected by volcanic activity. These additions improved both accessibility and user experience while preserving interactive performance (? , ?).

2.2.3 System Validation and Evaluation Metrics

To evaluate the effectiveness of the simulator, ? conducted quantitative and qualitative assessments. Visual fidelity was measured using the Structural Similarity Index (SSIM) and Root Mean Squared Error (RMSE) by comparing simulated eruption images with real-world photographs and surveillance footage of Taal Volcano. The study reported high visual similarity, indicating that the synthesized plume closely resembled actual volcanic eruptions. In addition, usability testing using the System Usability Scale (SUS) demonstrated that participants generally found the simulator easy to use and suitable for interactive exploration.

2.2.4 Architectural Limitations and Future Trajectories

Despite these accomplishments, the primary contribution of AnitoPlume lies in the design of its visualization framework rather than the underlying graphics API. The study focused on plume simulation, terrain modeling, visual fidelity, and usability evaluation, while the OpenGL implementation served primarily as the rendering platform supporting these objectives. Consequently, the original work did not investigate how the rendering architecture itself might influence performance, resource utilization, maintainability, or future extensibility.

The authors also acknowledged several opportunities for future development,

including the incorporation of more advanced rendering techniques, expanded physics-based simulations, compute shader implementations, and additional GPU-accelerated features. These recommendations highlight the need for a rendering architecture capable of supporting increasingly sophisticated graphics workloads while maintaining interactive performance.

2.3 Graphics API Architecture and Migration Implications

The modernization of AnitoPlume involves migrating the rendering pipeline from a legacy OpenGL based implementation to the Falcor rendering framework. Falcor utilizes DirectX12 as its underlying graphics API. This migration introduces a fundamentally different graphics architecture characterized by explicit resource management, reduced driver overhead, and improved utilization of modern multi-core processors. These architectural changes are particularly beneficial for AnitoPlume, whose real-time volcanic plume simulation continuously updates thousands of dynamic particle billboards while rendering large terrain models and high-resolution textures.

2.3.1 OpenGL Implicit State vs. DirectX 12 Explicit State

One of the defining characteristics of OpenGL is its use of an implicit global state machine. As described in Section 2.5 of the OpenGL specifications, the rendering state is stored within the current OpenGL context, where API commands modify persistent state that remains active until explicitly changed. Draw calls consume the current context state, including the active shader program, bound textures, vertex buffers, depth testing configuration, blending settings, and other rendering parameters. This design simplifies the use of the graphics library because rendering commands do not need to specify every state required for each execution. However, it introduces additional work for the graphics driver since it must determine which states have changed since the previous draw, validate that the current combination of states is valid, and translate the API calls into GPU commands. These operations contribute to increased CPU overhead particularly in applications with a large number of draw calls.

In contrast, DirectX 12 uses an explicit state management model. Rather than relying on a mutable global context, the application explicitly specifies the graphics pipeline configuration through Pipeline State Objects (PSOs), Root Sig-

natures, Descriptor Heaps, and Command Lists. A Pipeline State Object PSO encapsulates a complete graphics pipeline configuration including shaders, rasterizer state, depth and stencil settings, blending configuration, and input layout. The PSO is created and validated before rendering begins. During rendering, the application simply binds the appropriate PSO instead of modifying numerous individual rendering states.

This explicit approach substantially reduces driver overhead because state validation and pipeline compilation occur during PSO creation rather than at every draw call. As a result, the driver performs fewer runtime checks and can submit commands to the GPU more efficiently. Furthermore, DirectX 12 enables multiple CPU threads to record command lists independently, improving scalability on modern multi-core processors. Since each command list can be recorded independently without modifying a shared global rendering context, multiple CPU threads can generate rendering commands concurrently. Synchronization is required only when threads access shared application-managed resources, resulting in significantly less synchronization overhead than the implicit state model of OpenGL.

2.3.2 Exploding the CPU Command Recording Bottleneck

TODO: Expound more on how openGL limits CPU scalability

2.3.3 Explicit Memory Management & Resource Binding

TODO: Describe the differences between OpenGL's implicit GPU memory management and DirectX 12's explicit resource management model.

2.3.4 Falcor Rendering Framework

TODO: Explain how falcor abstracts the DX12 features through high level rendering interfaces. Explain how it speeds up development process while keeping the performance optimizations